Born-Oppenheimer Molecular Dynamics (BOMD) Trajectory Setup in GAUSSIAN 09

J. J. Radler,* C. Houferak,* and D. Lingerfelt*

Xiaosong Li Group, Department of Chemistry, University of Washington, Seattle, WA, USA

E-mail: jjradler@uw.edu; houferak@uw.edu; dblinger@uw.edu

Introduction

Born-Oppenheimer Molecular Dynamics (BOMD) is a computational modeling methodology that efficiently performs nuclear dynamics on a molecular system of interest. BOMD is implemented in the *Gaussian 09* software package, but steps to robustly sample the initial conditions (positions and momenta) from the NVT ensemble must be taken. This guide is intended for use with *Gaussian 09* and the keywords included here. This tutorial uses the simple molecule water at the **HF/STO-3G** level of theory, which is sufficiently small so as to run quickly on nearly any modern computing platform while still being illustrative.

Procedure

The procedure follows three main operations performed in *Gaussian*. First, the geometry of the molecule of interest is optimized so that the normal modes used in the next step are evaluated at the equilibrium geometry. Next, a number of very brief (2 steps) trajectories are computed in the thermal sampling step in which quanta of vibrational energy are deposited

in a Monte-Carlo fashion (subject to a Boltzmann distribution) into the normal modes and random phases are assigned to each of the populated modes. This process can be automated, but there is currently no standard script in *Gaussian* to do so, so this tutorial may be adapted into an automated process by the user if so desired.

Optimization

First we need an *ab initio* optimized geometry to feed into our thermal sampling calculation, so we create the input file `water_opt.com` with the following format and keywords.

```
#p HF/STO-3G opt

water_opt

0 1
O 0.00 0.00 0.00
H 1.00 0.00 0.00
H 0.00 1.00 0.00
```

where the standard keywords are applied and the file is terminated with the usual line of *whitespace*. Now we run the job by using the usual command

```
$ g09 water_opt.com &
```

where the ampersand trailing the command suppresses output and frees the `bash` terminal for other tasks while the process runs in the background. On a modern machine, this calculation should only take a few seconds.

Thermal Sampling

Obtain initial conditions which feed into the *Production MD*. This will be accomplished via populating approximate vibrational energy eigenstates in Monte-Carlo fashion according to the Boltzmann distribution. The vibrational modes that are populated in this procedure are the eigenfunction of the Taylor series expansion of the PES truncated at second order. So, iterative corrections to the initial condition sampled using this technique are required to ensure robust sampling of anharmonic modes. These corrections occur at "step 2" of the thermal sampling calculations, so we must request at least 2 steps (i.e. `maxpoints=2`) for each thermal sampling trajectory from which we will extract an initial condition for the production dynamics. The temperature for thermal sampling is set with `rtemp=298` which sets the sampling temperature at 298K (note this is omitted in the production calculation!)

To build the new input file, we first grab the optimized geometry from the output of the optimization job in `water_opt.log`. This can be done using `grep` in the command line, or it may be done manually in your favorite text editor. The block of Cartesian coordinates for the thermalized geometries should look like this:

Standard orientation:

Center Number	Atomic Number	Atomic Type	Coordinates (Angstroms) X	Y	Z
1	8	0	0.000000	0.000000	0.127159
2	1	0	-0.000000	0.758087	-0.508636
3	1	0	-0.000000	-0.758087	-0.508636

Once the extraneous characters are removed and the Cartesian coordinate matrix is formatted for another input file, we can build the input for the thermal sampling job. Our new input file for the thermal sampling job `water_sample.com` follows this format:

```
#p HF/STO-3G bomd(rtemp=298,maxpoints=3,ntraj=3)

water_sample

0 1
O   0.000000    0.000000    0.127159
H  -0.000000    0.758087   -0.508636
H  -0.000000   -0.758087   -0.508636
```

This input file will result in the generation of 3 initial conditions sampled from the NVT ensemble at 298K. To run the job we use the command:

```
$ g09 water_sample.com &
```

The geometries and velocities for initial conditions in *production MD* are found in the .log file immediately after the line

```
Scaling requirements satisfied.
```

However, the velocities here are not *mass weighted* (so if mass-weighting is important for your coordinate system, be aware of this. Be consistent!)

Production MD

This is the portion of the calculation that actually yields the time-dependent molecular properties. Three independent jobs using the three different initial trajectories may be parallelized by submitting the jobs independently to each processor (this is what we would call an *embarrassingly parallel* problem.) This may also be automated, but this tutorial goes through the process manually in order to provide a more transparent demonstration.

We can obtain the geometries and velocities for *each individual trajectory* by searching the log file for `Scaling requirements satisfied` and locating a block of text that looks like:

```
Rotational Energy =   0.4423070927D-02 Hartree
 CM Start Point Coordinates: bohr
 I=1 X= 0.000000000000D+00 Y= 2.400171627882D-03   Z= 1.523643140497D-01
 I=2 X= 0.000000000000D+00 Y= 1.379575918238D+00  Z= -1.193092352028D+00
 I=3 X= 0.000000000000D+00 Y= -1.417668383596D+00 Z= -1.225039871261D+00

 CM Start Point Velocity: bohr/s
 I=1 X= -4.423615155960D+12 Y= 8.776440047814D+12 Z= 2.254094642863D+12
 I=2 X= 8.574783786799D+13 Y= 1.526119315397D+13 Z=  4.040491397366D+13
 I=3 X= -1.554185545817D+13 Y= -1.545496651151D+14 Z= -7.617903158280D+13
```

After these lines have been modified in your favorite text editor, they can be turned into our input file for production MD runs. Each of these sets of positions and velocities corresponds to an *individual trajectory*, so in order to run multiple trajectories in production MD the coordinates must be extracted from each of the short trajectories in the thermal sampling output.

Our input file `water_production.com` should look like this:

```
#p hf/sto-3g bomd(gradientonly,maxpoints=1000) units=au iop(1/9=3)

water_production

! the following is the initial positions
0 1
O   0.000000000000D+00    2.400171627882D-03    1.523643140497D-01
H   0.000000000000D+00    1.379575918238D+00   -1.193092352028D+00
H   0.000000000000D+00   -1.417668383596D+00   -1.225039871261D+00

! the following is the initial velocities
0
  -4.423615155960D+12    8.776440047814D+12    2.254094642863D+12
   8.574783786799D+13    1.526119315397D+13    4.040491397366D+13
  -1.554185545817D+13   -1.545496651151D+14   -7.617903158280D+13
```

Make sure `UNITS=AU` as the output from the *thermal sampling* job is given in Bohr, and the default units for specifying coordinates in a Gaussian input file are Angstrom. Also, the `iop(1/9=3)` to specify the input velocities for the calculation in units of *Bohr/s*. Also

note that there must be a line of whitespace after both the position block, and also the initial velocities blocks. Also be aware that, while inline comments do not exist in *Gaussian* input files, we can include them after a `!`, but they will *not* count as whitespace, and the whitespace must still be included between the blocks. Also note the zero above the initial velocities block – we include this to indicate that there are to be no stopping criteria enforced for our trajectory (i.e. it will run intil the "maxpoints" time-steps have been taken in the simulation.) Finally, after running the simulation using this input file we see that we can `grep` the dipole moment vector for each time step using the command

```
$ grep Dipole water_production.log
```

which will output the following portion of the .log file (on the following page):

```
Dipole moment (field-independent basis, Debye):
Dipole        = 1.77727515D-02-6.65119565D-01 0.00000000D+00
Dipole        = 1.87643769D-02-6.64429795D-01-2.26517002D-04
Dipole        = 1.97514185D-02-6.63743592D-01-4.50848959D-04
Dipole        = 2.07338230D-02-6.63061210D-01-6.73033116D-04
Dipole        = 2.17115378D-02-6.62382900D-01-8.93107335D-04
Dipole        = 2.26845127D-02-6.61708905D-01-1.11111007D-03
Dipole        = 2.36527004D-02-6.61039467D-01-1.32708031D-03
Dipole        = 2.46160564D-02-6.60374820D-01-1.54105761D-03
Dipole        = 2.55745388D-02-6.59715197D-01-1.75308200D-03
Dipole        = 2.65281083D-02-6.59060824D-01-1.96319399D-03
Dipole        = 2.74767284D-02-6.58411922D-01-2.17143454D-03
Dipole        = 2.84203650D-02-6.57768711D-01-2.37784505D-03
Dipole        = 2.93589867D-02-6.57131403D-01-2.58246728D-03
Dipole        = 3.02925648D-02-6.56500206D-01-2.78534341D-03
Dipole        = 3.12210727D-02-6.55875325D-01-2.98651596D-03
Dipole        = 3.21444866D-02-6.55256959D-01-3.18602775D-03
Dipole        = 3.30627851D-02-6.54645304D-01-3.38392197D-03
Dipole        = 3.39759492D-02-6.54040550D-01-3.58024204D-03
Dipole        = 3.48839621D-02-6.53442884D-01-3.77503170D-03
Dipole        = 3.57868094D-02-6.52852487D-01-3.96833490D-03
Dipole        = 3.66844793D-02-6.52269538D-01-4.16019585D-03
Dipole        = 3.75769618D-02-6.51694210D-01-4.35065898D-03
Dipole        = 3.84642495D-02-6.51126672D-01-4.53976889D-03
Dipole        = 3.93463370D-02-6.50567088D-01-4.72757039D-03
Dipole        = 4.02232210D-02-6.50015619D-01-4.91410846D-03
Dipole        = 4.10949006D-02-6.49472421D-01-5.09942820D-03
```

```
 Dipole        = 4.19613766D-02-6.48937647D-01-5.28357489D-03
 Dipole        = 4.28226521D-02-6.48411444D-01-5.46659392D-03
 Dipole        = 4.36787323D-02-6.47893956D-01-5.64853078D-03
 Dipole        = 4.45296240D-02-6.47385323D-01-5.82943108D-03
 Dipole        = 4.53753363D-02-6.46885681D-01-6.00934051D-03
 Dipole        = 4.62158801D-02-6.46395160D-01-6.18830483D-03
```

This output time series of dipoles from the .log file may be modified and used to calculate simulated IR spectra. The output file also includes a time series of polarizabilities (if requested via the appropriate keywords), which may be extracted using the same `grep` command and used to compute Raman and vibrational spectra.